

Grafana — Operator Manual

What this technology does

Grafana is the read-only visualization layer for telemetry, infrastructure state, and gateway-evidence status. It helps an operator see what the system is doing; it is **not** the authoritative source of research measurements or verification decisions.

Current commissioned boundary

- Grafana runs on ULC-03.
- Current commissioned cloud build uses Grafana 13.2.0.
- PostgreSQL datasource uses the read-only `telemetry_reader` role.
- Telemetry and gateway-evidence panels are provisioned.
- Counted Chapter IV results come from sealed recorder/run evidence, not screenshots or dashboard refreshes.

Step 1 — Check the Grafana runtime

On ULC-03:

```
if [ -d /opt/grafana ]; then
  cd /opt/grafana
  sudo docker compose ps
else
  echo 'GRAFANA_PATH_NOT_FOUND=/opt/grafana'
fi
```

PASS means: the Grafana service/container is running without a restart loop.

If the path has deliberately changed, find the current Compose deployment from the cloud runbook/repository before creating a second Grafana instance.

Step 2 — Check the local HTTP health endpoint

```
curl --connect-timeout 3 --max-time 5 -fsS http://127.0.0.1:3000/api/health
```

If the current cloud deployment binds a different approved private/loopback address, use that documented endpoint.

PASS means: Grafana returns healthy database/version information.

Do not expose port 3000 publicly just to make this check easy. Use the documented SSH/private access path.

Step 3 — Check recent logs

```
cd /opt/grafana
sudo docker compose logs --since=15m --tail=150 grafana
```

Look for datasource TLS/auth failures, provisioning errors, plugin crashes, database locks, or restart loops.

Step 4 — Verify the PostgreSQL datasource independently

Before editing a dashboard because it is blank, use a protected read-only database connection and run:

```
SELECT now() AT TIME ZONE 'UTC' AS db_utc;

SELECT time, dev_eui, metric_name, metric_value, metric_text, metric_bool, unit, quality
FROM telemetry.measurements
ORDER BY time DESC
LIMIT 20;
```

PASS means: the database has current rows and the read-only path works.

If SQL is healthy but Grafana is blank, the problem is now Grafana/query/time-range/provisioning—not ingestion.

Step 5 — Verify Grafana really uses read-only credentials

The datasource must use `telemetry_reader` or the current reviewed equivalent. It must not use:

- PostgreSQL administrator/superuser;
- `telemetry_writer`;
- gateway-evidence verifier writer;
- Fabric adapter database identity;
- OpenBao token/AppRole;
- Fabric client identity.

A dashboard must never need write authority.

Step 6 — Verify time range and freshness before declaring data missing

For every operational dashboard, check:

1. browser/Grafana time zone;
2. selected dashboard time range;
3. latest database event timestamp;
4. panel query interval;
5. datasource query result;
6. panel transformation/field mapping.

A stale panel is different from a failed ingestion path. Display freshness explicitly rather than making the operator infer it from a graph shape.

Step 7 — Verify telemetry panels

A useful sensor panel should make the following obvious when those values exist:

- device/source identity;
- measurement name;
- value and human-readable unit;
- timestamp/age;
- RSSI/SNR/link quality where appropriate;
- battery status;
- no-data/stale state.

Avoid labels such as `unitless`, unexplained percentages, or raw machine field names when a human-readable unit/meaning is available.

Step 8 — Verify evidence panels separately from telemetry freshness

Evidence dashboards may display states such as:

```
pending
verified
evidence_gap
integrity_failure
```

They must not reinterpret a non-verified status as verified.

Keep separate views for:

- telemetry freshness;
- gateway checkpoint/evidence freshness;
- pending verification age;
- verification outcome counts;
- Fabric outbox/reconciliation state.

A fresh sensor row with stale evidence is a real degraded evidence condition, not a completely healthy event.

Step 9 — Fabric/outbox display rule

When displaying `telemetry.fabric_outbox`, distinguish at minimum:

```
pending
processing
failed
reconciling
confirmed
needs_attention
dead_letter
```

`submitted_unknown` may exist only as legacy migration compatibility. Do not present it as the current normal uncertain-submit state.

A pending row with `attempts=0` does not automatically mean Fabric is down. The dashboard/query should expose enough context to distinguish missing `finalized_payload` or unverified v2 evidence from an adapter-processing

failure.

Safe dashboard/provisioning change procedure

1. verify current Grafana health;
2. verify the datasource directly with SQL;
3. back up/export current dashboard/provisioning state;
4. change one dashboard/query/provisioning item;
5. reload/restart Grafana only if the changed mechanism requires it;
6. verify the panel against a known SQL result;
7. verify time range, unit, freshness, no-data behavior;
8. verify datasource remains read-only;
9. verify counted research results are still sourced from the recorder, not Grafana.

Do not modify Node-RED ingestion or database rows to make a chart look nicer.

Troubleshooting map

Symptom	First check
Grafana URL unavailable	container/listener/local access path
datasource test fails	PgBouncer/DB/TLS/read-only credential
SQL has rows, panel blank	dashboard time/query/variable/transformation
panel values stale	latest DB timestamp vs dashboard range/refresh
wrong unit/label	panel field config/query alias, not sensor ingestion
evidence status confusing	verifier tables/query semantics
dashboard differs from counted result	recorder/run evidence is authoritative

Backup/recovery

Preserve the deployed Compose/config/provisioning/dashboard state and Grafana persistent data according to the cloud backup runbook. Restoring Grafana must not require changing the telemetry database or granting write privileges.

After restore, verify:

1. local /api/health;
2. datasource connection;
3. one known telemetry query;
4. one evidence-status query;
5. dashboard units/freshness.

Completion checklist

- Grafana on ULC-03 is running;

- local health endpoint passes;
- datasource uses a read-only role;
- direct SQL returns expected telemetry;
- dashboards show identity, units, timestamp/freshness, and no-data state clearly;
- evidence status is displayed without changing it;
- Fabric queue states are not oversimplified;
- research results remain sourced from sealed recorder evidence.

Detailed guides

- [../server/integrations/grafana/00-README.md](#)
- [../server/cloud-production/14a-grafana-cloud-deployment.md](#)
- [../server/cloud-production/14-observability-alerting-and-logging.md](#)

Supporting runbook: 00-README.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Grafana Monitoring for ChirpStack Telemetry

Use these guides to connect Grafana to the separate PostgreSQL + TimescaleDB telemetry database, build useful dashboards, and configure alerts.

Run Docker and database commands on the LoRaWAN application server from /opt/lorawan-lab. Perform Grafana menu and panel actions in a browser connected through the documented SSH tunnel. The browser reaches Grafana; it does not connect directly to PostgreSQL.

```
ChirpStack MQTT
-> Node-RED telemetry flow
-> PostgreSQL / TimescaleDB
    |
    +-> Grafana PostgreSQL data source
    |     -> dashboards, variables, alerts
    |
    +-> optional gateway_evidence verification state
        -> Grafana reads status/freshness only
```

Grafana is not a LoRaWAN network server and does not receive RF packets. It visualizes data after ChirpStack has accepted, decrypted, decoded, and stored a device uplink. A rendered value is not automatically current or trustworthy; every operational dashboard must expose timestamp, freshness, and no-data state.

When gateway integrity is enabled, Grafana may also display verifier-owned pending, verified, evidence_gap, and integrity_failure state plus checkpoint freshness. Grafana never creates or changes that verification result.

Values used in this guide

Component	Value
Grafana container	grafana
Grafana UI	Prefer http://127.0.0.1:3000 through an SSH tunnel; use direct LAN binding only when approved
PostgreSQL service	telemetry-db
Database	lorawan_telemetry
Grafana database role	telemetry_reader
Telemetry schema	telemetry
Main table	telemetry.uplinks
Optional evidence schema	gateway_evidence, only after the reviewed integration is deployed

Component	Value
Evidence access	Read-only approved status/checkpoint views or tables; never raw evidence mutation

Guide map

File	Purpose
01-install-and-connect.md	Add Grafana, configure the PostgreSQL data source, and secure first login
02-build-dashboards.md	Build latest-value, trend, fleet, link-quality, and battery panels
03-alerting-and-operations.md	Create alert rules, contact points, variables, and operating procedures
04-security-backup-and-updates.md	Protect accounts, tokens, dashboards, and upgrades
05-troubleshooting.md	Diagnose data-source, query, time, and dashboard failures

Final checks

- Grafana uses telemetry_reader, never the database administrator, Node-RED writer, gateway-evidence verifier writer, OpenBao role, or Fabric adapter identity.
- The PostgreSQL data source reports a successful connection.
- A real uplink appears in Explore.
- A dashboard displays temperature, humidity, battery, RSSI, and SNR when available.
- Dashboard variables can switch between devices.
- Alerts have documented thresholds and no-data behavior.
- When v2 is enabled, dashboards distinguish telemetry freshness from evidence freshness and do not display pending/gap/failure as verified.

Next: [01-install-and-connect.md](#)

Supporting runbook: 14a-grafana-cloud-deployment.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

14A. Tiny Grafana POC on ulc-03

Status: SERVER-ONLY STAGING + EVIDENCE OBSERVABILITY COMPLETE / PASS; REAL-DATA ACCEPTANCE HARDWARE DEFERRED. Grafana 13.2.0 is running on ulc-03 from immutable digest sha256:3fd54ae1214669f8355f065ec9f6445d5279a3d77095ab048ca045685272429b, loopback-only on 127.0.0.1:3000, with strict-TLS telemetry_reader datasource health and read-only ACL proof PASS. The provisioned LoRaWAN Telemetry Overview now has six panels: the original four telemetry panels plus Evidence checkpoints and Evidence verification state, both using approved read-only gateway_evidence views. The six-panel file was loaded by Grafana provisioning without a restart. GRAFANA_SERVER_STAGING=PASS remains authoritative; full Phase 14A still requires a real EMU-01 payload-v2 row and freshness/latest-reading correlation. Grafana is not part of the LoRaWAN control path.

Grafana is only a **small visualization aid** for the HA POC. It is not part of the LoRaWAN control path and it gets no dedicated Droplet.

14A.1 Data path

```
Gateway / device
  |
  v
ChirpStack
  |
  v
Node-RED
  |
  v
lorawan_telemetry database
  |
  v
Grafana
```

The database is the same three-member Patroni PostgreSQL cluster used by ChirpStack. lorawan_telemetry has TimescaleDB enabled and stores the telemetry hypertables; there is no separate TimescaleDB server in this POC.

14A.2 Resource target

Keep the workload small, but do not impose a hard limit below Grafana's published installation minimum:

```
Host CPU:      1 shared vCPU; do not add a 0.25-CPU hard cap
Memory limit:  512 MiB
Refresh:       60 seconds
```



```
Users:      about 1 operator during the POC
Dashboards: only the few panels needed for the test
```

Grafana documents 512 MB memory and 1 CPU core as its minimum recommended installation resources. The POC host has only one **shared** vCPU and also runs other services, so this remains an experimental placement rather than a production Grafana sizing claim. Removing the 0.25-CPU cap lets Grafana burst when a dashboard loads instead of creating an artificial CPU bottleneck.

If Grafana reaches the 512-MiB ceiling or materially harms the ha-03 failure test, record the measurement and resize based on evidence. Do not remove other required architecture features merely to preserve dashboards.

14A.3 Preconditions

For **server-only staging now**, require:

- Patroni is healthy with one primary and two replicas;
- lorawan_telemetry exists with the commissioned telemetry schema/hypertables;
- telemetry_reader exists with read-only permissions and authenticates through the local PgBouncer/HAProxy path;
- ulc-03 has enough free memory/disk for the 512-MiB Grafana ceiling;
- TCP/3000 is free before first start;
- an exact Grafana image version/digest and its runtime UID/GID are inspected before creating persistent-data ownership;
- the PgBouncer public CA is copied to a Grafana-readable **CA-only** path without loosening /etc/lorawan-pki/pgbouncer.

The following is a **final Phase 14A acceptance prerequisite**, not a blocker for server staging:

- Node-RED has stored at least one real **EMU-01 payload-v2** row and duplicate safety has been proven for that event.

An empty query result before hardware returns is expected. The staging gate proves the secure Grafana-to-database path; the later real-reading gate proves telemetry freshness and dashboard semantics.

Recorded server-only preflight + image pin PASS — 2026-08-29

The ulc-03 preflight completed without starting Grafana:

```
TCP/3000 = FREE
host memory = 1.9 GiB total / about 1.2 GiB available
swap = 0
root disk = about 34 GiB available
Node-RED A = healthy, restart_count=0
Grafana running = NO
```

Observed colocated container memory at this checkpoint was approximately 64.6 MiB for Node-RED A, 31.13 MiB for OpenBao, 164.6 MiB for Spilo, and 19.06 MiB for etcd. This is a preflight snapshot, not a permanent resource guarantee.

The commissioned PgBouncer CA source remained protected as /etc/lorawan-pki/pgbouncer/ca.crt (root:postgres, mode 0640) and SHA-256 matched:

```
6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb
```

Grafana image pin:

```
release: 13.2.0
immutable image:
  grafana/grafana@sha256:3fd54ae1214669f8355f065ec9f6445d5279a3d77095ab048ca045685272429b
commit:
  f681b1359f6a0b8ecb9f2c49a88ac72b75bde73b
runtime UID: 472
runtime GID: 0
runtime groups: 0
/var/lib/grafana in image: 472:0 mode 777
```

GRAFANA_SERVER_PREFLIGHT=PASS and GRAFANA_SERVER_PREFLIGHT_EXIT=0 are authoritative. The image pull/inspection did not create a Grafana container or listener.

telemetry_reader credential boundary

Repository review found no approved plaintext telemetry_reader credential location. PgBouncer contains a valid SCRAM-SHA-256 verifier, but the verifier is intentionally non-reversible and must not be scraped or treated as a password.

Before creating the Grafana data source, either retrieve the existing plaintext from an approved external secret store/custodian or deliberately rotate telemetry_reader through PostgreSQL plus all three PgBouncer userlists, verifying each endpoint before staging the replacement password only in Grafana's protected host secret file. Do not grant Grafana telemetry_writer, weaken SCRAM/TLS, or place the plaintext in this repository.

Immediate next boundary: resolve that credential custody/rotation decision, then create the Grafana filesystem/public-CA/protected-env/Compose layer from the exact image pin above and keep the first listener loopback-only.

telemetry_reader controlled-rotation preflight PASS — 2026-08-29

The read-only Phase 14A credential preflight on ulc-03 passed without changing PostgreSQL, PgBouncer, Grafana, or any HA service. Patroni role discovery returned exactly one writable leader: ulc-01 / 10.104.0.2 with /leader = 200, while ulc-02 / 10.104.0.4 and ulc-03 / 10.104.0.8 returned /replica = 200. Local ulc-03 PgBouncer remained active/enabled on 10.104.0.8:6432. PostgreSQL reported telemetry_reader|true|scram, proving the role is LOGIN-enabled with a SCRAM-SHA-256 verifier. The protected PgBouncer auth file remained 0640 root:postgres, contained exactly four entries, and exactly one entry was telemetry_reader. Grafana remained absent and TCP/3000 remained free.

The first userlist line-count command used `sudo wc -l < /etc/pgbouncer/userlist.txt`; the invoking non-root shell attempted the redirection before `sudo` and correctly received `Permission denied`. This was a command-harness issue only, not a PgBouncer permission defect. The corrected privileged count returned exactly 4.

GRAFANA_TELEMETRY_READER_ROTATION_PREFLIGHT=PASS. The next bounded mutation is only to generate a new high-entropy telemetry_reader password into a root-only pending file on ulc-03. Do not alter the PostgreSQL role until that pending file's ownership, mode, length, and format have been verified without printing the secret.

telemetry_reader replacement-secret preparation PASS — 2026-08-29

The replacement reader credential was generated only on ulc-03 and stored at /root/grafana-bootstrap/telemetry-reader-password.pending. The file is 0600 root:root, exactly 65 bytes, and validated as 64 lowercase hexadecimal characters plus one trailing newline without printing the credential.

GRAFANA_TELEMETRY_READER_SECRET_PREP=PASS. PostgreSQL and all PgBouncer nodes were still unchanged at this checkpoint.

To reduce operator time without weakening the gate structure, subsequent rotation commands may be grouped into a single child-shell block only when the block verifies each boundary before advancing: rediscover leader -> prove superuser TLS path -> rotate only telemetry_reader -> prove direct new-password TLS login -> wait for verifier replication -> regenerate and validate one PgBouncer userlist -> reload in place -> prove new-password authentication through that PgBouncer endpoint. The script must abort at the first failed checkpoint and must never print the plaintext password or SCRAM verifier.

telemetry_reader PostgreSQL rotation + ulc-03 PgBouncer refresh partial PASS — 2026-08-29

The streamlined gated block completed the credential mutation through the ulc-03 PgBouncer refresh. Patroni was rediscovered immediately before mutation with exactly one leader, ulc-01 / 10.104.0.2; the existing Spilo superuser verify-full path to that leader returned postgres|10.104.0.2|f. ALTER ROLE telemetry_reader PASSWORD ... succeeded without printing the password. A fresh direct verify-full login with the protected replacement credential returned telemetry_reader|lorawan_telemetry|f, proving the new PostgreSQL credential itself is valid. The new SCRAM verifier replicated to ulc-03 immediately, the regenerated four-role PgBouncer candidate passed role/count/SCRAM checks, and the previous auth file was preserved at /etc/pgbouncer/userlist.txt.before-telemetry-reader-20260829T010150Z. The installed userlist remained 0640 root:postgres; PgBouncer reloaded in place with PID 789143 unchanged. These checkpoints are authoritative PASS and must not be repeated merely because the following client probe failed.

The first disposable-client probe stopped **before PgBouncer/SCRAM authentication** because psql could not read /run/pgbouncer/ca.crt inside that disposable container. The retry changed only the client harness: the exact current Spilo image was run as numeric 0:0, the mounted CA was proven readable and byte-identical to commissioned SHA-256 6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb, and a verify-full login through physical endpoint 10.104.0.8:6432 using logical host pgbouncer.internal.lorawan.com returned telemetry_reader|lorawan_telemetry|f. CONTAINER_CA_READABLE=PASS and ULC03_ROTATED_CREDENTIAL_END_TO_END=PASS are authoritative. No database or service state changed during the retry. Do not repeat the PostgreSQL rotation or the ulc-03 userlist reload.

telemetry_reader three-node rotation COMPLETE / PASS — 2026-08-29

The remaining two PgBouncer verifier refreshes completed sequentially and safely. ulc-02 regenerated the authoritative four-role SCRAM userlist from its replicated PostgreSQL state, proved all non-reader verifiers unchanged, preserved /etc/pgbouncer/userlist.txt.before-telemetry-reader-20260829T010816Z, installed the new 0640 root:postgres file, and reloaded PgBouncer in place with PID 787012 unchanged. A verify-full login from ulc-03 through physical endpoint 10.104.0.4:6432 returned telemetry_reader|lorawan_telemetry|f.

Only after ulc-02 passed, ulc-01 repeated the same bounded refresh, preserving /etc/pgbouncer/userlist.txt.before-telemetry-reader-20260829T010820Z and keeping PgBouncer PID 1105819 unchanged. Verify-full authentication through 10.104.0.2:6432 returned the same expected role/database/writable-primary result. A final all-three gate then authenticated the rotated telemetry_reader credential successfully through 10.104.0.2:6432, 10.104.0.4:6432, and 10.104.0.8:6432, while Patroni still showed exactly one leader on 10.104.0.2 and replicas on .4 and .8.

THREE_NODE_PGBOUNCER_READER_AUTH=PASS, FINAL_PATRONI_SINGLE_LEADER=PASS, and GRAFANA_TELEMETRY_READER_SECRET_PROMOTION=PASS are authoritative. The protected password was promoted only after the three-endpoint gate to /root/grafana-bootstrap/telemetry-reader-password, remaining 0600 root:root and 65 bytes. Do not repeat this rotation unless a future authentication failure or deliberate credential-rotation policy requires it.

Grafana filesystem + trust + protected-config staging PASS — 2026-08-29

The no-start staging block on ulc-03 completed successfully. /etc/lorawan-cloud/grafana is 0750 root:root; /srv/grafana/data is 0700 with numeric owner 472:0, matching the pinned Grafana runtime; /etc/lorawan-

pki/grafana-pgbouncer is 0750 root:root; and its public ca.crt is 0640 root:root with the commissioned SHA-256 6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb. The protected /etc/lorawan-cloud/grafana/grafana.env is 0600 root:root, contains the immutable image pin, a generated Grafana admin credential, and the rotated telemetry_reader credential without printing either secret. /etc/lorawan-cloud/grafana/compose.yml is 0640 root:root, validates successfully, binds only 127.0.0.1:3000:3000, mounts the dedicated CA copy at /run/pgbouncer/ca.crt, and maps pgbouncer.internal.lorawan.com to 10.104.0.8.

A disposable container using the exact pinned Grafana image and numeric 472:0 proved the data directory writable and the CA readable and byte-identical. Final safety checks still found no Grafana container and TCP/3000 free. GRAFANA_HOST_DIRECTORIES=PASS, GRAFANA_PGBOUNCER_CA_COPY=PASS, GRAFANA_PROTECTED_ENV=PASS, GRAFANA_COMPOSE_VALIDATION=PASS, GRAFANA_RUNTIME_FILESYSTEM_ACCESS=PASS, and GRAFANA_RUNTIME_CA_ACCESS=PASS are authoritative.

Grafana 13.2 plugin immutability note: current Grafana documentation still treats PostgreSQL as bundled/no-install in standard deployments, while the 13.2 source's preinstall registry includes grafana-postgresql-datasource and Grafana's generic preinstalled-plugin auto-update setting defaults on. For this immutable POC, set GF_PLUGINS_PREINSTALL_AUTO_UPDATE=false before first start so a preinstalled plugin cannot drift independently of the pinned server image. Record the observed PostgreSQL plugin identity/version after startup. Provisioning continues to use the supported type: postgres alias.

The next boundary is datasource/dashboard provisioning plus the first controlled Grafana start and runtime verification.

14A.4 Minimal container

Run on ulc-03. Create a dedicated directory instead of adding Grafana to an unrelated Compose project. Do not choose ownership for /srv/grafana/data until the pinned image's numeric runtime UID/GID has been inspected:

```
sudo install -d -m 750 /etc/lorawan-cloud/grafana
# create /srv/grafana/data after the image UID/GID gate
cd /etc/lorawan-cloud/grafana
```

Pin GRAFANA_IMAGE to an exact tested immutable digest and keep the admin password plus the telemetry_reader password in a mode-600 environment file outside Git. Do not place either password in Compose or Markdown. Create /etc/lorawan-pki/grafana-pgbouncer/ca.crt as a public CA-only copy of the commissioned PgBouncer CA; the source /etc/lorawan-pki/pgbouncer remains protected for the PgBouncer service and must not be made traversable by Grafana.

```
services:
  grafana:
    image: ${GRAFANA_IMAGE}
    restart: unless-stopped
    mem_limit: 512m
    ports:
      - "127.0.0.1:3000:3000"
    environment:
      GF_SECURITY_ADMIN_USER: ${GRAFANA_ADMIN_USER}
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_ADMIN_PASSWORD}
      GF_USERS_ALLOW_SIGN_UP: "false"
      GF_AUTH_ANONYMOUS_ENABLED: "false"
    volumes:
      - /srv/grafana/data:/var/lib/grafana
      - /etc/lorawan-pki/grafana-pgbouncer/ca.crt:/run/pgbouncer/ca.crt:ro
    extra_hosts:
      - "pgbouncer.internal.lorawan.com:10.104.0.8"
```

Do not add a Prometheus stack just to satisfy this POC. Command-line HA checks plus a telemetry dashboard are enough unless infrastructure metrics are part of a specific test.

14A.5 Start and open it

```
docker compose config --quiet
docker compose up -d grafana
docker compose ps grafana
docker compose logs --since=5m --tail=100 grafana
sudo ss -lntp | grep ':3000'
```

Expected listener:

```
127.0.0.1:3000
```

From the admin workstation:

```
ssh -L 3000:127.0.0.1:3000 <USER>@<HA03_MANAGEMENT_IP>
```

Open <http://127.0.0.1:3000> locally.

14A.6 PostgreSQL data source

Use the local HA database path on ha-03:

```
Host: pgbouncer.internal.lorawan.com:6432
Database: lorawan_telemetry
User: telemetry_reader
TLS/SSL: enabled
SSL mode: verify-full (or the equivalent strict hostname+CA mode exposed by the pinned Grafana version)
Root CA: /run/pgbouncer/ca.crt
```

The extra_hosts mapping sends this logical name to ha-03's local PgBouncer. Verify the data source with **Save & Test** before building a dashboard. If verification fails, fix the CA/name; do not disable certificate verification as the permanent solution.

Grafana must not use telemetry_admin, telemetry_writer, fabric_adapter, or the ChirpStack database role.

Why: Grafana only reads a few rows for demonstration.

14A.7 First dashboard

Keep it tiny:

```
latest uplink time
reading age
sensor temperature
sensor pressure
RSSI
SNR
last 20 uplinks
```

Only show fields that the device actually sends.

Use the actual generic telemetry schema. For the latest normalized measurements:

```
SELECT
    time,
```

```
dev_eui,  
metric_name,  
metric_value,  
metric_text,  
metric_bool,  
unit,  
quality  
FROM telemetry.measurements  
ORDER BY time DESC  
LIMIT 20;
```

For the primary EMU-01 dashboard, select values by approved metric_name, for example barometer_pressure_pa, barometer_temperature_c, environment_temperature_c, environment_humidity_percent, the two distinct light metrics, soil, UV, rain, and battery. Do not assume a dedicated SQL column exists for every sensor. SEC-02's temporary RAK12011 verification payload is not the permanent dashboard baseline.

14A.8 Healthy-state commissioning verification

Do **not** stop Grafana during setup. Instead verify the normal read-only path:

```
Grafana container = running  
listener = 127.0.0.1:3000 only  
data source connects as telemetry_reader through ulc-03 PgBouncer/HAProxy  
one real Node-RED-stored sensor row is visible  
latest-reading age is visible  
at least one short historical trend panel is visible  
no write-capable database credential is configured  
healthy-state memory use is recorded
```

The intended failure behavior remains: if ulc-03 or Grafana is unavailable, visualization may pause while the LoRaWAN core and persisted PostgreSQL telemetry survive. **Phase 15 tests that claim; Phase 14A does not inject it.**

Server-only staging PASS boundary

While the gateway is unavailable, Grafana server staging is complete when all of these pass without requiring a telemetry row:

```
exact immutable Grafana image/digest recorded  
container runtime UID/GID recorded  
persistent data ownership matches that runtime identity  
admin credential stored only in protected host configuration  
127.0.0.1:3000 is the only Grafana listener  
Grafana starts healthy without a restart loop  
pgbouncer.internal.lorawan.com resolves to 10.104.0.8 inside Grafana  
/run/pgbouncer/ca.crt is readable and byte-identical to the commissioned PgBouncer CA  
PostgreSQL data source authenticates as telemetry_reader with strict TLS verification  
read-only schema/catalog query succeeds even when telemetry tables contain zero rows  
dashboard/provisioning definition is saved for later real-data validation
```

Record this as GRAFANA_SERVER_STAGING=PASS, **not** PHASE14A=PASS.

Server-only Grafana activation COMPLETE / PASS — 2026-08-29

The first controlled Grafana start on ulc-03 completed the full server-only staging boundary. Grafana started from the pinned 13.2.0 image and reached /api/health after 23 seconds with version 13.2.0, commit f681b1359f6a0b8ecb9f2c49a88ac72b75bde73b, runtime state running|0|false, and the expected 512 MiB container memory limit. TCP/3000 is exposed only as 127.0.0.1:3000; no public Grafana listener was created. The runtime also confirmed GF_PLUGINS_PREINSTALL_AUTO_UPDATE=false.

Inside Grafana, `pgbouncer.internal.lorawan.com` resolves to `10.104.0.8` and `/run/pgbouncer/ca.crt` remains byte-identical to commissioned SHA-256 `6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb`. The provisioned PostgreSQL data source resolved at runtime to `grafana-postgresql-datasource`, authenticated as `telemetry_reader`, targeted `lorawan_telemetry` through `pgbouncer.internal.lorawan.com:6432`, and returned `datasource health OK with sslmode=verify-full`. Grafana's own data-source query proved `telemetry_reader` has schema usage plus `SELECT` on `telemetry.uplinks` and `telemetry.measurements`, while `INSERT` remains denied on both. The empty-table baseline was valid: `uplink_rows=0` and `measurement_rows=0`.

The observed PostgreSQL plugin identity/version is `grafana-postgresql-datasource 13.0.1`. The file-provisioned LoRaWAN Telemetry Overview dashboard loaded successfully with four panels: latest uplink/RF quality, reading age, latest selected measurements, and last 20 uplinks. Node-RED A remained `running|0|healthy`; Patroni remained exactly one leader on `10.104.0.2` plus replicas on `.4` and `.8`. Healthy-state container snapshot at this checkpoint was Grafana about `286.8 MiB / 512 MiB` and Node-RED about `75.77 MiB / 1.922 GiB`.

`GRAFANA_SERVER_STAGING=PASS` is now authoritative. Do **not** mark full `PHASE14A=PASS` yet because no real EMU-01 row exists at this server-only checkpoint.

Synthetic application read-path checkpoint - COMPLETE / PASS with fresh fixture; cleanup pending

Node-RED server commissioning created one intentionally synthetic all-zero-DevEUI payload-v2 fixture and independently proved its telemetry/outbox replay semantics. The first Grafana-own-datasource attempt on 2026-08-30 confirmed Grafana `13.2.0`, commit `f681b1359f6a0b8ecb9f2c49a88ac72b75bde73b`, loopback-only runtime, provisioned `datasource UID lorawan-telemetry` using `telemetry_reader` against `pgbouncer.internal.lorawan.com:6432` with `sslmode=verify-full`, and dashboard `UID lorawan-overview` with the four expected panels. The captured API verifier then stopped after parsing zero base rows.

Later forensics proved that the fixture was not lost through Grafana, Patroni, replication, or Timescale retention: all three members are consistent on timeline 3 with zero lag, Patroni history predates the event, and telemetry has zero Timescale jobs. PostgreSQL statistics show the exact corresponding delete counts (`uplinks=1`, `measurements=13`, `fabric_outbox=1`); ordinary application identities lack `DELETE`. `u1c-03` shell history contains the targeted cleanup SQL, while the primary `u1c-01` sudo journal records the matching privileged `psql -XAtq cleanup` invocation at 2026-08-29 15:35:09 UTC. `SYNTHETIC_ROWSET_CLEANUP_ATTRIBUTED=PASS`. The evidence does not establish why that cleanup invocation occurred despite the captured API failure, so do not claim a specific shell-control-flow mechanism.

A fresh fixture `grafana-synthetic-GRAFANA-SYNTH-20260830T000012Z` completed that remaining server-only Grafana data proof. It was inserted from a clean reserved-identity baseline through the exact deployed Node-RED Function and immediately appeared as exactly 1 uplink, 13 measurements, and 1 pending outbox row. Grafana stayed healthy at the existing 512-MiB ceiling. The actual provisioned dashboard targets returned `P1=1`, `P2=1`, `P3=6`, `P4=1` rows, and a code-mode `datasource query` executed as `telemetry_reader` returned the exact event, all thirteen measurements, the pending outbox row, matching `test_sequence=1788048012`, reading age 5 seconds, and correct latest-view counts. Final database counts remained `1|13|1`; no cleanup ran in the proof block. Evidence is under `/home/opsadmin/lorawan-ha-evidence/GRAFANA-SYNTH-20260830T000012Z`.

`GRAFANA_SYNTHETIC_FIXTURE_AND_READ_PATH=PASS` is authoritative for the server-only Grafana read path.

A separate cleanup-only block then re-proved the exact `1|13|1` fixture and its safe pending/unclaimed/unsealed outbox state, discovered exactly one Patroni primary (`10.104.0.2`), and removed only that event in one guarded transaction. The local replica immediately reached `0|0|0`; the reserved synthetic identity returned to `0|0|0`; all three PostgreSQL members independently returned `0|0|0`. Grafana stayed `running|0|false|536870912`, Node-RED A stayed `running|0|healthy`, B remained fenced, and the prior Grafana evidence manifest remained valid while a separate cleanup manifest was added. `GRAFANA_SYNTHETIC_CLEANUP_COMPLETE=PASS` is authoritative. This synthetic result still cannot satisfy the real-sensor freshness requirement below.

Hardware-dependent final acceptance

After a real EMU-01 payload-v2 event has passed Node-RED/TimescaleDB commissioning, verify the expected dashboard panels show that event, the displayed `test_sequence/timestamp` correspond to the database row, and the freshness/latest-reading-age panel behaves correctly. Only then close the full Phase 14A pass condition.

14A.9 Pass condition

- no extra monitoring Droplet exists;
- Grafana stays loopback-only on `u1c-03`;
- it uses only `telemetry_reader`;
- its TLS-verified data source reads `lorawan_telemetry` through PgBouncer/HAProxy;
- real Node-RED-stored sensor values and a short history render correctly;
- the dashboard exposes freshness/latest-reading age needed by the later acceptance tests;
- healthy-state resource use is recorded without OOM evidence.

Next required pre-test setup: [20-openbao-and-fabric-adapter.md](#), despite its higher file number. Do not begin Phase 15 yet.

Supporting runbook: 14-observability-alerting-and-logging.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

14. Minimal POC Observability and Evidence Capture

Status: 14S SERVER-ONLY HARNESS PRE-STAGING MAY PROCEED / FINAL PHASE 14 AFTER 13B. The final healthy-baseline evidence run still occurs only after Phase 13B against the fully commissioned system. While hardware/provider/Fabric dependencies are unavailable, an interim **14S server-only harness** may be run against the already commissioned server components to prove commands, permissions, UTC timestamps, redaction, and evidence-directory structure early. 14S is preparation evidence, not the final Phase 14 PASS.

The POC needs **repeatable evidence**, not a production monitoring platform.

Do not deploy Prometheus, Alertmanager, Loki, or another monitoring stack merely to prove the three-server architecture. For a few sensors, command-line checks, service logs, a small Grafana telemetry dashboard, and timestamped evidence folders are enough.

The important rule is:

First prove the evidence harness on a fully healthy baseline. Only after Phase 14B passes should the same capture set be used before, during, and after Phase 15 failures.

Phase 14S server-only evidence-harness pre-stage

Before the final Phase 13B exists, the server-only portion of the harness may be exercised once with a SERVER-PRESTAGE-... run ID. Populate only a before/ state; do not create a fake failure state. Prove non-secret capture for the components that are actually deployed: host resources, etcd, Patroni/PostgreSQL/TimescaleDB, PgBouncer/HAProxy, Valkey/Sentinel, Mosquitto, ChirpStack, Node-RED A/B state, Grafana, OpenBao, and telemetry.fabric_outbox. Record hardware/provider/Fabric-adapter/gateway-integrity fields as DEFERRED or BLOCKED rather than inventing values.

The 14S pass marker is SERVER_ONLY_EVIDENCE_HARNESS=PASS. This means the server-side commands/permissions are ready. It does **not** authorize Phase 15 and does not replace the final healthy-baseline run.

Phase 14S current execution state - 2026-08-29

The first server-only healthy-state capture created run ID SERVER-PRESTAGE-20260829T133354Z under /home/opsadmin/lorawan-ha-evidence/SERVER-PRESTAGE-20260829T133354Z. It is read-only: no service restart, configuration mutation, backup, or failure injection is part of this run.

The following gates are authoritative PASS and must not be repeated merely because a later harness step stopped:

```
THREE_HOST_EVIDENCE=PASS
ETCD_3_MEMBER_HEALTH=PASS
PATRONI_1_LEADER_2_REPLICAS=PASS
POSTGRES_TIMESCALE_OUTBOX=PASS
NODE_RED_SINGLE_ACTIVE=PASS
```

At capture time Patroni reported 10.104.0.2 as the single leader and 10.104.0.4 / 10.104.0.8 as replicas. Node-RED A was running|0|healthy with HTTP 200 on 127.0.0.1:1880; Node-RED B remained fenced/stopped. The recurring PostgreSQL locale warning remained non-blocking.

The first Grafana step printed GRAFANA_CONTAINER=FAIL, but this was a discovery-harness false negative rather than a service failure. The harness searched Docker's displayed image text for the word grafana; because the immutable image was displayed as digest prefix 3fd54ae12146, no candidate was found. A direct read-only diagnostic then proved the Compose-labeled container grafana was running, restart count 0, OOMKilled=false, memory limit 536870912, listener 127.0.0.1:3000, and /api/health returned database ok, Grafana 13.2.0, commit f681b1359f6a0b8ecb9f2c49a88ac72b75bde73b. GRAFANA_RUNTIME_DISCOVERY=PASS is authoritative.

The corrected resume preserved Steps 1-6, then passed GRAFANA_HEALTH=PASS using the Compose service label / loopback port rather than image-name text, and passed CHIRPSTACK_TWO_NODE_RUNTIME=PASS on ulc-01 and ulc-02. These are now authoritative and must not be rerun merely because the next OpenBao probe stopped.

The OpenBao step then failed before collecting any member state because docker exec openbao bao status - format=json used the CLI default https://127.0.0.1:8200. The commissioned OpenBao listeners are intentionally bound to the host-private addresses 10.104.0.2:8200, 10.104.0.4:8200, and 10.104.0.8:8200; container loopback is not a commissioned listener. The resulting dial tcp 127.0.0.1:8200: connect: connection refused is therefore a harness-address defect, not evidence of OpenBao service failure. Resume only from OpenBao onward and set BAO_ADDR=https://<that-node-private-ip>:8200 plus BAO_CACERT=/openbao/tls/ca.crt for every in-container bao status call.

The final resume set BAO_ADDR explicitly to each commissioned private OpenBao API address and BAO_CACERT=/openbao/tls/ca.crt. All three members returned initialized=true, sealed=false, and ha_enabled=true, so OPENBAO_3_NODE_HEALTH=PASS is authoritative. The harness then wrote the provider/hardware/Fabric/gateway-integrity items as DEFERRED or BLOCKED, passed the evidence secret-sanity scan, normalized the evidence tree to mode 0700 directories / 0600 files, and generated a final SHA-256 manifest covering the complete 16-file evidence set. DEFERRED_BLOCKED_RECORD=PASS, EVIDENCE_SECRET_GATE=PASS, EVIDENCE_SHA256=PASS, and EVIDENCE_FILESYSTEM_PROTECTION=PASS are authoritative.

SERVER_ONLY_EVIDENCE_HARNESS=PASS is therefore complete for run SERVER-PRESTAGE-20260829T133354Z. The evidence directory is /home/opsadmin/lorawan-ha-evidence/SERVER-PRESTAGE-20260829T133354Z. No service restart, configuration mutation, backup, or failure injection occurred during this 14S run. Do not repeat 14S merely to continue setup or begin a new chat; the final Phase 14 healthy-baseline evidence capture still occurs after final Phase 13B against the fully commissioned system.

Phase 14 healthy-baseline dry run

After the final Phase 13B snapshot exists, create a normal evidence directory and populate the before/ state for every deployed component. Do not create a fake during/ failure state. Verify all commands run non-interactively with the intended least-privilege access, timestamps are UTC, and no secrets are copied into evidence.

The final dry run must include Node-RED, Grafana, OpenBao, both Fabric adapters, the latest normal Fabric commit, the current Reserved-IP owner, and the final Phase 13B backup-set identifier.

14.1 What every Phase 15 failure run must answer

For each run be able to answer:

```
what failed?
when was the fault injected?
which PostgreSQL / Valkey / OpenBao roles changed?
did etcd / Sentinel / OpenBao quorum survive?
which Droplet owned the Reserved IPv4 before/after failure?
did the public-ingress failover agent move it, and how long did that take?
which MQTT broker served after failure?
which ChirpStack instance remained usable?
did a NEW post-fault real uplink succeed?
did Timescale telemetry remain consistent?
did the Fabric outbox remain durable?
was Fabric adapter execution deployed or BLOCKED?
did any 2-GiB host OOM or become swap-bound?
what was the measured recovery time?
was full 3-node redundancy restored before the next run?
```

14.2 Create one evidence directory per test

On the administration workstation or another approved evidence host:

```
RUN_ID="HA-$(date -u +%Y%m%dT%H%M%S)-<TEST>-R1"
EVIDENCE="$HOME/lorawan-ha-evidence/$RUN_ID"
umask 077
mkdir -p "$EVIDENCE"/{before,during,after,logs}
printf '%s\n' "$RUN_ID" | tee "$EVIDENCE/RUN_ID.txt"
date -u --iso-8601=seconds | tee "$EVIDENCE/created-at.txt"
```

Create notes.md containing only non-secret facts:

```
run ID:
failure being tested:
fault method:
staging Gateway EUI:
staging DevEUI:
pre-fault test_sequence/event key:
Reserved IPv4 owner before fault:
Reserved IPv4 owner after recovery:
Reserved-IP reassignment action/time:
fault timestamp UTC:
first successful post-fault event UTC:
RTO:
unexpected behavior:
restore timestamp UTC:
final result: PASS / FAIL / BLOCKED
```

Do not copy passwords, private keys, AppKeys, bearer tokens, complete DSNs, or OpenBao recovery material into the evidence folder.

14.3 Host resource snapshot

Run on **ha-01**, **ha-02**, and **ha-03** before/during/after the test and save the output under the matching evidence subdirectory:

```
hostname
printf '\n=== UTC ===\n'
date -u --iso-8601=seconds
printf '\n=== MEMORY ===\n'
free -h
printf '\n=== CPU / LOAD ===\n'
uptime
command -v vmstat >/dev/null && vmstat 1 5 || true
printf '\n=== DISK ===\n'
```

```
df -h /
printf '\n=== SERVICES ===\n'
systemctl --failed --no-pager
printf '\n=== CONTAINERS ===\n'
docker stats --no-stream 2>/dev/null || true
printf '\n=== OOM ===\n'
journalctl -k --since today --no-pager | grep -Ei 'oom|out of memory|killed process' || true
```

Why: with 2-GiB shared-CPU nodes, an OOM **or a functional failover failure caused by sustained CPU contention** is a sizing result. Do not restart the process and erase the evidence before recording it. Use the functional recovery criteria rather than inventing a universal CPU-utilization threshold.

14.4 etcd snapshot

From an approved administrative shell, using the currently tested east-west endpoints:

```
ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  endpoint health

ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  endpoint status --write-out=table

ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  member list --write-out=table
```

These commands match the current HTTP-only etcd deployment on 10.104.0.0/20. Add CA/client options only after an etcd TLS rollout has been completed and tested.

Record:

```
healthy member count
leader/member IDs
revision
3/3 before
2/3 during one-host/member loss
3/3 after restore
```

A 2/3 state is acceptable only **during the single-failure test**. Do not begin another fault until it is back to 3/3.

14.5 PostgreSQL / Patroni snapshot

Use the exact pinned Patroni configuration:

```
patronictl -c <PATRONI_CONFIG> list <PG_SCOPE> --extended
```

Then verify the normal client route from the host being tested:

```
psql \
  'host=pgbouncer.internal.lorawan.com port=6432 dbname=postgres user=<MONITOR_ROLE> sslmode=verify-full' \
  -c 'SELECT now() AT TIME ZONE '\''UTC'\'' AS utc_time, inet_server_addr(), pg_is_in_recovery();'
```

Expected for the routed server:

```
pg_is_in_recovery = false
```

Record:

```
primary before/after
replica members
lag where available
promotion/switchover time
server address returned through PgBouncer/HAProxy
```

Both logical databases must remain present:

```
chirpstack
lorawan_telemetry
```

14.6 TimescaleDB snapshot

Against lorawan_telemetry through the normal PgBouncer path:

```
psql \
'host=pgbouncer.internal.lorawan.com port=6432 dbname=lorawan_telemetry user=telemetry_reader
sslmode=verify-full' \
-c "SELECT extname, extversion FROM pg_extension WHERE extname='timescaledb';"

psql \
'host=pgbouncer.internal.lorawan.com port=6432 dbname=lorawan_telemetry user=telemetry_reader
sslmode=verify-full' \
-c "SELECT hypertable_schema, hypertable_name FROM timescaledb_information.hypertables ORDER BY 1,2;"
```

Required hypertables:

```
telemetry.uplinks
telemetry.measurements
```

Also record the latest real measurement without assuming a sensor-specific SQL column:

```
SELECT time, dev_eui, metric_name, metric_value, metric_text, metric_bool, unit, quality
FROM telemetry.measurements
ORDER BY time DESC
LIMIT 20;
```

After PostgreSQL promotion, repeat the extension/hypertable queries. A promoted member that cannot load TimescaleDB is not an acceptable HA recovery.

14.7 PgBouncer / HAProxy snapshot

On each host that owns PgBouncer:

```
SHOW POOLS;
SHOW SERVERS;
SHOW STATS;
```

Use the protected PgBouncer admin/stats DSN and redact credentials from saved output.

Record:

```
client waiting count
server connection count
```

```
server address before/after promotion
unexpected auth failures
reconnect behavior
```

For the PostgreSQL HAProxy route, a read-only SQL query through :15432 must reach a writable primary. For MQTT/Valkey/OpenBao, use the component-specific checks below rather than assuming an HAProxy process means every backend is healthy.

14.8 Valkey / Sentinel snapshot

Check all three data nodes:

```
valkey-cli --tls --cacert <VALKEY_CA> \
-h <VALKEY_PRIVATE_IP> -p 6379 \
-a '<LOAD_FROM_PROTECTED_SOURCE>' ROLE
```

Check all three Sentinels:

```
valkey-cli --tls --cacert <VALKEY_CA> \
-h <SENTINEL_PRIVATE_IP> -p 26379 \
-a '<LOAD_SENTINEL_SECRET_PROTECTED>' \
SENTINEL CKQUORUM lorawan-valkey

valkey-cli --tls --cacert <VALKEY_CA> \
-h <SENTINEL_PRIVATE_IP> -p 26379 \
-a '<LOAD_SENTINEL_SECRET_PROTECTED>' \
SENTINEL GET-MASTER-ADDR-BY-NAME lorawan-valkey
```

Finally query the **application endpoint**:

```
valkey-cli --tls --cacert <VALKEY_CA> \
--sni valkey.internal.lorawan.com \
-h <LOCAL_HAPROXY_PRIVATE_IP> -p 16379 \
-a '<LOAD_FROM_PROTECTED_SOURCE>' ROLE
```

Record:

```
primary before/after
replica count
Sentinel CKQUORUM result
promotion time
HAProxy endpoint role
```

The application endpoint must always report the writable primary/master.

14.9 MQTT snapshot

The cloud port ownership must be:

```
HAProxy public gateway service:      :8883 on ulc-01/ulc-02 anchors
HAProxy ChirpStack workload service:  :18883 on ulc-01/ulc-02 private VPC
HAProxy Node-RED ingest service:      :18884 on ulc-03 private VPC
Mosquitto gateway-facing TLS:         :8884 on ulc-01/ulc-02
Mosquitto Node-RED mTLS:              :8886 on ulc-01/ulc-02
Mosquitto ChirpStack workload TLS:    :8885 on ulc-01/ulc-02
Gateway-local buffer:                 127.0.0.1:1883 on Raspberry Pi only
```

On cloud hosts:

```
sudo ss -lntp | grep -E ':(8883|8884|8885|18883|18884)\b' || true
```

Record from HAProxy/Mosquitto logs:

```
preferred broker health
backup broker health
gateway disconnect/reconnect timestamp
Node-RED/ChirpStack MQTT reconnect if affected
first successful post-switch uplink
```

On the Raspberry Pi also record the local queue/persistence indicators described by the gateway buffer manual before, during, and after a backhaul/broker test.

Do not call successful reconnection proof of broker-session replication. The POC is testing service failover plus the gateway's bounded persistent uplink buffer.

14.10 ChirpStack snapshot

Record:

```
ChirpStack-1 process/HTTP health
ChirpStack-2 process/HTTP health
public https://chirpstack.<DOMAIN> result
gateway last-seen
OTAA result when that test is selected
fresh uplink event ID / sequence
downlink result when that test is selected
```

A running process is not enough. The recovery timer ends only on the first **new post-fault** staging-device uplink accepted through the surviving path, using the definition in [15-failover-chaos-and-acceptance-testing.md](#).

14.10A Node-RED and Grafana snapshot

On ulc-03, capture:

```
Node-RED container state/restart count
Node-RED listener = 127.0.0.1:1880 only
Node-RED private MQTT route :18884 listener/backend health
latest real telemetry event_key/test_sequence stored through Node-RED
Node-RED exported flow hash/reference
Grafana container state/restart count
Grafana listener = 127.0.0.1:3000 only
Grafana datasource identity = telemetry_reader
latest dashboard-visible reading time/age
Grafana dashboard export hash/reference
```

Use a read-only database query to correlate the latest telemetry row with the dashboard. Do not use the Grafana admin credential as evidence that the database path works.

14.11 OpenBao snapshot

From an approved OpenBao admin environment:

```
bao status
bao operator raft list-peers
```

Through the stable HAProxy endpoint:

```
curl --fail --silent --show-error \
  --cacert /etc/lorawan-pki/openbao/ca.crt \
  'https://openbao-kms.internal.<DOMAIN>:18200/v1/sys/health?standbyok=true'
```

Record:

```
member count
active node
sealed/unsealed state
healthy Raft peer set = 3/3
Transit sign/verify result through stable :18200
current Transit key-version metadata without private key material
Phase 15 will record 2/3 behavior during one-member loss
```

Do not put root tokens, recovery shares, or unseal material in the evidence folder.

14.12 Fabric outbox and conditional adapter evidence

The outbox is a durable application boundary and must be monitored independently from adapter process health.

Record status counts with a read-only database role using the deployed schema:

```
SELECT status, count(*)
FROM telemetry.fabric_outbox
GROUP BY status
ORDER BY status;
```

Also record oldest eligible/pending age using the actual schema timestamp column documented by the outbox migration.

Current commissioned adapter evidence

Record:

```
ULC-01 adapter-1 enabled state + process health + worker_id
ULC-02 adapter-2 write-disabled/fenced state
outbox eligibility: finalized_payload present; v2 verifier status=verified
pending / processing / failed / confirmed counts
reconciling count + oldest age
needs_attention count + oldest age
legacy submitted_unknown count only as migration compatibility
live/expired lease evidence
OpenBao HA/sign/verify health
HRC Gateway reachability and TLS identity
Fabric tx ID + HRC record ID + commit/reconcile result for sampled work
```

Do not infer that pending with attempts=0 is a Fabric outage. First prove the row satisfies the current claim predicate. Do not report ULC-02 as an active writer merely because its container/image exists.

During an external Fabric outage the expected pattern is:

```
telemetry still inserts
outbox grows/waits
```


14.12A Gateway-integrity evidence-service capture

Add this section only after the reviewed v2 evidence services are actually deployed. It is distinct from the generic test evidence folder: these files prove the **security lineage services themselves**.

Capture at minimum:

```
gateway-journal.txt
  journal version/executable hash
  process/restarts
  last sequence/record hash
  open/closed segment IDs
  storage usage/reserve
  unuploaded segment count/bytes
  uploader retry state
  latest accepted checkpoint receipt age

evidence-storage.txt
  backend/path identity without secrets
  free space/capacity
  object count/oldest backlog where meaningful
  no-overwrite/append-only configuration evidence

evidence-ingest.txt
  ingest-1 + ingest-2 process/version/image digest
  per-replica health/restart/OOM/resource state
  listener/SNI/backend identity
  accepted/rejected/conflicting upload counters
  latest accepted checkpoint/segment receipt
  duplicate retry convergence to one identity
  unknown/unauthenticated-client rejection evidence

mqtt-evidence-collector.txt
  collector-1 + collector-2 process/version/image digest
  four persistent broker sessions: each collector -> broker-1 + broker-2
  dedicated client IDs/identity names without secrets
  subscription scope
  collector lag/latest captured gateway event
  deterministic capture_key convergence
  publish/command permission denial result

evidence-verifier.txt
  verifier-1 + verifier-2 process/version/image digest
  identical trusted-decoder digest
  current worker_id/lease state
  oldest pending work
  pending/verified/evidence_gap/integrity_failure counts
  unmatched journal/MQTT/application counts
  checkpoint conflict count
  trusted-decoder mismatch count
  expired-lease reclaim evidence from commissioning/Phase15 as appropriate

trusted-decoder.txt
  decoder_id/version or code hash
  fixed-vector self-test result
  latest raw_app_data_sha256/normalized_digest_sha256 references for the selected event

gateway-evidence-state.txt
  selected source event key/observed_at
  verification_id/status/reason
  journal segment/sequence/hash references
  checkpoint ID
  MQTT gateway_event_id
  verified_at/update age

checkpoint-freshness.txt
```

```
latest accepted checkpoint per gateway
sequence/segment and server_received_at
age relative to current UTC
```

Do not copy raw private keys, bearer tokens, unrestricted payload dumps, or OpenBao/Fabric credentials into the Phase 14 folder. Raw segments/events remain in the protected evidence store; Phase 14 records stable IDs, hashes, status, and bounded sanitized excerpts.

Diagnostic rule:

```
telemetry fresh + MQTT fresh + checkpoint stale
```

means delivery is healthy while the evidence upload/ingest path is degraded. Keep those states separate.

14.13 Grafana

Grafana is visual confirmation, not the source of truth for failover timing.

Use the small dashboard for:

```
latest sensor value
latest timestamp / reading age
RSSI/SNR where stored
small history view
```

Keep a slow refresh interval appropriate to the few-sensor POC. Do not change the 2-GiB resource profile by running aggressive dashboard polling during the sizing test.

14.14 Before / during / after capture pattern

For each component, save the same command output three times:

```
<run-id>/
before/
  hosts-*.txt
  public-ingress.txt
  etcd.txt
  patroni.txt
  timescale.txt
  pgbouncer-*.txt
  valkey-sentinel.txt
  mqtt.txt
  chirpstack.txt
  openbao.txt
  outbox.txt
  gateway-journal.txt           # when v2 deployed
  evidence-storage.txt         # when v2 deployed
  evidence-ingest.txt          # when v2 deployed
  mqtt-evidence-collector.txt  # when v2 deployed
  evidence-verifier.txt        # when v2 deployed
  trusted-decoder.txt          # when v2 deployed
  gateway-evidence-state.txt   # when v2 deployed
  checkpoint-freshness.txt     # when v2 deployed
during/
  same files
after/
  same files
logs/
  only the bounded/sanitized excerpts needed to explain the transition
notes.md
```

Screenshots may supplement the folder, but do not replace machine-readable command output.

14.15 Pass condition

Observability setup is ready only when a **healthy-state dry run** produces, without undocumented memory:

```
UTC/run identifier and Phase 13B backup ID
host CPU/memory/swap/OOM evidence
etcd 3/3
Patroni primary/replica/lag state
PgBouncer pool/wait state
Timescale extension/hypertable/latest telemetry state
Valkey/Sentinel primary/quorum state
MQTT backend + HAProxy listener state
ChirpStack-1/2 health + gateway last-seen
Reserved IPv4 current owner + failover timer state
Node-RED state + current flow hash + latest stored event
Grafana state + dashboard hash + latest visible reading age
OpenBao 3/3 + Transit normal-path result
when v2 is selected: journal/uploader + ingest + evidence storage + MQTT collector + verifier + trusted
decoder + checkpoint freshness + verification-state evidence
when v2 is selected: one real staging lineage already reached verified before any Fabric v2 seal
Fabric adapter-1/2 + outbox counts + last confirmed tx
```

The harness must also have commands/templates ready to populate the later during/ and after/ folders, but **no fault is injected in Phase 14**.

Production monitoring, centralized logs, long retention, alert routing, and SLO dashboards are later hardening, not prerequisites for this small architecture.

Next: [14b-pre-test-commissioning-gate.md](#). Phase 15 remains blocked until that gate passes.

Canonical source: deployment/technology-guides/11-grafana.md. For any mutable production-status statement, check the dated baseline and verified live state.